

HttpSession

👉 Session Creation & Management:

- The HttpSession object allows you to store user-specific data (like name and city) across multiple requests.
- **Creating/Accessing Session:** Use `request.getSession()` to create a new session if one does not exist or to access the current session.
- **Session Attributes:** You can store attributes using `session.setAttribute("key", value)` and retrieve them with `session.getAttribute("key")`.

👉 Redirecting Between Servlets:

- **Storing Data in Session:** Data stored in one servlet can be accessed in another servlet by the same user session. This is useful when redirecting between different parts of the application.
- **Redirecting:** Use `response.sendRedirect("URL")` to redirect the user to another servlet while keeping the session data intact.

👉 Accessing Session Data:

- **Retrieving Existing Session:** Use `request.getSession(false)` in the second servlet to retrieve the current session without creating a new one if it doesn't exist.
- **Accessing Attributes:** Retrieve the stored attributes using `session.getAttribute("key")` and use them as needed.

```
@WebServlet("/FirstServletApp")
public class FirstServletApp extends HttpServlet {
    public void service(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException
    {
        String name = request.getParameter("uname");
        String city = request.getParameter("ucity");

        // Creating or accessing the session
        HttpSession session = request.getSession();
        session.setAttribute("name", name); // Storing data in session
        session.setAttribute("city", city);

        // Redirecting to another servlet
        response.forward ("SecondServletApp");
    }
}
```

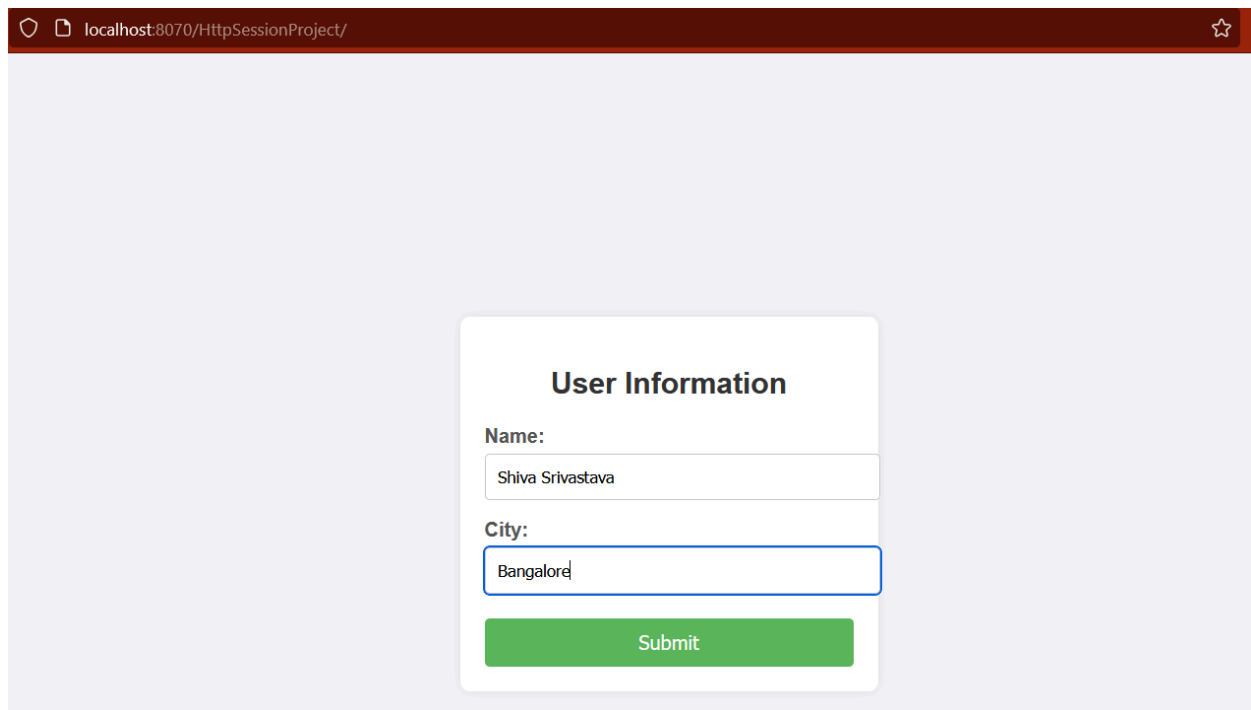
```
@WebServlet("/SecondServletApp")
public class SecondServletApp extends HttpServlet {
    public void service(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException
    {
        // Accessing the current session without creating a new one
        HttpSession session = request.getSession(false);

        String name = (String) session.getAttribute("name"); //
Retrieving session data
    }
}
```

```
String city = (String) session.getAttribute("city");

// Displaying the session data
PrintWriter pw = response.getWriter();
pw.println("<h1>Name: " + name + "</h1>");
pw.println("<h1>City: " + city + "</h1>");
pw.close();
}
}
```

Output:



The screenshot shows a web browser window with the address bar displaying 'localhost:8070/HttpSessionProject/'. The main content area has a light purple background. In the center, there is a white rounded rectangle titled 'User Information'. Inside this rectangle, there are two labels: 'Name:' and 'City:'. Below 'Name:' is a text input field containing 'Shiva Srivastava'. Below 'City:' is a text input field containing 'Bangalore'. At the bottom of the white rectangle is a green button with the text 'Submit'.

Name is Shiva Srivastava

City is Bangalore

👉 **Summary:**

- Use HttpSession to manage user data across multiple servlets.
- Store data with session.setAttribute(), retrieve it using session.getAttribute(), and share it across servlets even after redirection.
- Always ensure the session is retrieved correctly (getSession(false)) when accessing it in subsequent servlets to avoid creating a new session unnecessarily.